

TipJar+ – Kompleksowy Plan Architektury i Realizacji Projektu

Wprowadzenie i Wizja Projektu

TipJar+ to platforma mikropłatności Web3, która umożliwi fanom łatwe wysyłanie napiwków (donacji) ulubionym twórcom w formie stablecoina USDC. Celem projektu jest zaoferowanie twórcom nowego źródła przychodu przy minimalnych opłatach i globalnym zasięgu, wykorzystując technologię blockchain do wyeliminowania tarcia i pośredników. Dzięki stablecoinowi powiązanemu z USD (USDC) unika się zmienności kryptowalut, a jednocześnie uzyskuje szybkie, transgraniczne transfery bez tradycyjnych opłat bankowych. Napiwki o nawet niewielkiej wartości stają się opłacalne, gdyż koszty transakcyjne są niemal zerowe – to zachęca do kultury częstego wspierania twórców drobnymi kwotami ("buy me a coffee"). Platforma łączy cechy serwisów takich jak Patreon czy Twitch (bezpośrednie wsparcie dla twórców) z zaletami blockchain (błyskawiczne płatności bez granic i samodzielna kontrola środków przez użytkowników).

Wartość dla użytkowników: Fani mogą błyskawicznie tipować swoich ulubionych streamerów czy influencerów (nawet jako goście, bez zakładania konta), a twórcy otrzymują niemal całość wsparcia – TipJar+ planuje pobierać jedynie ok. 7% (3,5% przy zasileniu + 3,5% przy wypłacie) plus ewentualne opłaty wypłaty, co jest znacznie mniejszą prowizją niż na tradycyjnych platformach (OnlyFans zatrzymuje ~20%, Patreon ~5% + opłaty płatności, Twitch jeszcze więcej). Dzięki niskim opłatom i eliminacji minimalnych prowizji nawet napiwek rzędu kilku centów jest opłacalny – to może zbudować kulturę częstego, drobnego wspierania twórców na całym świecie.

Partnerstwo z Circle: Strategicznym partnerem technologicznym projektu jest firma Circle, dostarczająca fundamentalną infrastrukturę finansową – custodial wallets, płatności on-ramp/off-ramp, Gas Station oraz Paymaster. Usługi takie jak Circle Programmable Wallets (Developer-Controlled Wallets), Payments API, Gas Station i Payouts API stanowią kręgosłup operacyjny TipJar+, umożliwiając realizację kluczowych funkcjonalności platformy (automatyczne portfele dla użytkowników, szybkie transfery USDC wewnątrz systemu, sponsorowanie opłat gas, wypłaty na zewnętrzne konta itp.). Dzięki bliskiej integracji z Circle, TipJar+ może skupić się na doświadczeniu użytkownika, opierając warstwę finansową na sprawdzonych rozwiązaniach.

W kolejnych sekcjach przedstawiono architekturę systemu oraz szczegółowy plan realizacji, rozbitą na podzadania i checklisty dla wszystkich procesów. Dokument ma służyć jako kompleksowy przewodnik dla zespołu wdrożeniowego – od ogólnej wizji po najdrobniejsze zadania do wykonania – tak aby każdy członek zespołu rozumiał pełen obraz projektu i kolejność działań. Na końcu zarysowano też dalszą wizję rozwoju platformy poza MVP, zgodnie z długoterminową koncepcją SocialFi.

Architektura Systemu

TipJar+ zostało zaprojektowane jako nowoczesna aplikacja webowa typu klient-serwer z wykorzystaniem chmury. System jest wyraźnie podzielony na frontend i backend, które komunikują się poprzez zdefiniowane API. Taki podział ułatwia skalowanie oraz niezależny rozwój interfejsu użytkownika i logiki serwerowej. Poza głównymi komponentami aplikacji, w

ekosystemie funkcjonują usługi zewnętrzne, przede wszystkim platforma Circle zapewniająca infrastrukturę finansową (custodial wallets, płatności, sponsorowanie gazu). Ponadto TipJar+ integruje dostawców tożsamości OAuth (Google, Twitch) dla uproszczenia logowania oraz korzysta z sieci blockchain (np. Polygon jako domyślna sieć obsługi USDC) i rozwiązań Account Abstraction (np. bundler ERC-4337 Pimlico) z myślą o przyszłych ulepszeniach.

Hosting i infrastruktura: Aplikacja frontendowa (Next.js) jest hostowana na Vercel dla zapewnienia globalnej dostępności, automatycznych deployów, CDN oraz serverless funkcji (API routes). Backend (NestJS) oraz baza danych PostgreSQL są uruchomione w chmurze AWS (np. w regionie UE na potrzeby zgodności z RODO), gdzie można je skalować horyzontalnie (np. kontenery na ECS Fargate lub Kubernetes, za Load Balancerem) i korzystać z zarządzanych usług (RDS Postgres, Secrets Manager do przechowywania kluczy itp.). Całość projektowano z myślą o bezpieczeństwie (brak przechowywania wrażliwych danych kart po stronie TipJar, hasła tylko hashowane, zabezpieczenia przed CSRF i atakami typu XSS/SQLi, ograniczenia rate limiting, WAF itp.) oraz o dobrym UX (responsywne UI, szybkie ładowanie dzięki SSR i cache, płynne interakcje, proste flow płatności).

Komponenty systemu: Poniższy diagram przedstawia główne komponenty architektury TipJar+ i ich powiązania:

Schemat architektury TipJar+ – frontend i backend komunikują się przez REST API, zaś backend integruje się z zewnętrznymi usługami (Circle, OAuth). Baza danych przechowuje dane użytkowników i transakcji. Portfele użytkowników (DCW) oraz płatności obsługiwane są poprzez API Circle.

Front-end aplikacji to webowa aplikacja Next.js (React + TypeScript), która dostarcza interfejs użytkownika. Mamy dwa główne tryby działania: widok fana (np. strona profilowa twórcy z widżetem do wysłania napiwku) oraz panel twórcy (dashboard do zarządzania profilem i środkami). Next.js pozwala na Server-Side Rendering i statyczne generowanie stron, co wykorzystujemy dla lepszego SEO publicznych profili twórców oraz szybszego wczytywania. Stylowanie oparte jest o Tailwind CSS zgodnie z opracowanym design systemem (kolory złoty/turkusowy, itp.), z dodatkiem dynamicznych elementów (np. animacje przy przejściach, podpowiedzi) dla atrakcyjnego UX. Aplikacja korzysta z globalnego store (biblioteka Zustand) do przechowywania stanu (informacje o zalogowanym użytkowniku, saldo portfela, historia transakcji itp.), co ułatwia komunikację między komponentami React.

Back-end to aplikacja NestJS (Node.js + TypeScript) uruchomiona na AWS. Jest to wielomodularny serwer API REST, odpowiadający za logikę biznesową, bezpieczeństwo oraz integracje zewnętrzne. Backend udostępnia zestaw endpointów REST chronionych mechanizmem JWT (JSON Web Token) dla frontendu. Kluczowe moduły backendu to m.in.: moduł uwierzytelniania (Passport.js z strategiami OAuth2 dla Google/Twitch oraz obsługa Web3 SIWE, generowanie tokenów JWT), moduł użytkowników/profilu, moduł portfeli i integracji z Circle, moduł obsługi płatności (logika zasileń fiat→USDC, obsługa Paymastera do opłat gazowych), moduł transakcji (ewidencja tipów) oraz moduł powiadomień (wysyłanie notyfikacji w czasie rzeczywistym). Backend komunikuje się z bazą danych PostgreSQL przy pomocy ORM Prisma – baza przechowuje dane o użytkownikach (w tym poświadczenia

OAuth/SIWE), profilach twórców, powiązanych identyfikatorach portfeli Circle, ewidencję transakcji (tipów, depozytów, wypłat) oraz ustawienia aplikacji.

Integracja z Circle: Backend pełni rolę pośrednika między naszą aplikacją a usługami finansowymi Circle. Wykorzystujemy oficjalny SDK @circle-fin/developer-controlled-wallets do komunikacji z API Circle w zakresie zarządzania portfelami i płatnościami. Dla każdego twórcy, przy rejestracji automatycznie tworzony jest dedykowany Circle Developer Controlled Wallet (DCW) typu Smart Contract Account – portfel powiązany z naszym projektem, zarządzany przez backend TipJar. Identyfikator tego portfela (circleWalletId) oraz główny adres blockchain SCA są bezpiecznie przechowywane w bazie danych. Dzięki wykorzystaniu portfeli typu SCA już od początku, TipJar+ jest gotowy do integracji z usługą Circle Gas Station – portfele SCA umożliwiają sponsorowanie opłat transakcyjnych, co w przyszłości pozwoli wyeliminować dla użytkowników problem płacenia za gaz (o czym dalej).

Circle zapewnia dwie kluczowe ścieżki przepływu środków w TipJar+: wewnętrzne transfery off-chain oraz transakcje on-chain. W ramach ekosystemu Circle możliwe są natychmiastowe, bezprowizyjne transfery między portfelami DCW użytkowników (np. gdy fan z portfelem TipJar przesyła napiwek twórcy – odbywa się to jako wewnętrzny zapis w systemie Circle, bez transakcji on-chain i bez opłat). Gdy jednak zachodzi potrzeba interakcji on-chain (np. wypłata środków na zewnętrzny portfel Ethereum/Polygon lub napiwek bezpośrednio z portfela MetaMask fana), Circle może zrealizować transakcję on-chain i – w naszej konfiguracji – zasponsorować opłatę gas dzięki mechanizmowi Gas Station/Paymaster. TipJar+ planuje wykorzystać Circle Gas Station do pokrywania opłat sieciowych za transakcje inicjowane z naszych portfeli DCW (zarówno przy wypłatach, jak i ewentualnych działaniach on-chain z poziomu backendu). Natomiast Circle Paymaster posłuży do umożliwienia fanom korzystającym z własnych portfeli (EOA, np. MetaMask) opłacania gazu w USDC – czyli faktycznie eliminacji potrzeby posiadania przez nich ETH/MATIC, co zapewni bezbolesne doświadczenie (gasless).

Inne usługi zewnętrzne: W warstwie frontend przewidziano integrację z portfelami kryptowalutowymi użytkowników – np. przycisk "Połącz portfel Web3" inicjuje `window.ethereum.request({ method: 'eth_requestAccounts' })` (MetaMask) i tworzy providera Ethers.js do podpisywania transakcji w ramach mechanizmu Sign-In with Ethereum (SIWE). Proces SIWE pozwala użytkownikom logować się bez hasła, przy pomocy klucza prywatnego ich portfela (szczegóły w sekcji logowania). Poza tym TipJar+ korzysta z dostawców OAuth (Google, Twitch) dla tradycyjnych metod logowania, a także z usług takich jak Sentry (monitoring błędów na produkcji) czy biblioteka QRCode (generacja kodów QR z linkiem do profilu twórcy).

Kluczowe Procesy i Funkcjonalności

W ramach TipJar+ zidentyfikować można kilka kluczowych procesów biznesowych oraz odpowiadające im funkcjonalności, które zostaną zaimplementowane. Poniżej opisujemy te procesy wraz z podziałem na kroki i zadania potrzebne do ich realizacji. Każdy proces został zaprojektowany tak, aby maksymalnie uprościć doświadczenie użytkownika, jednocześnie zapewniając niezawodność i bezpieczeństwo.

Onboarding Twórców i Uwierzytelnianie Użytkowników

Rejestracja Twórcy: Proces rozpoczyna się od założenia konta przez nowego twórcę. TipJar+ umożliwia rejestrację twórców za pomocą e-maila i hasła (tradycyjny signup) oraz poprzez konto Google (OAuth 2.0). Po stronie backendu zaimplementowany jest moduł Auth oparty na NestJS Passport – obsługuje on tworzenie nowych kont, szyfrowanie haseł, logowanie oraz federacyjne logowanie przez Google (a w kolejnym etapie także Twitch i SIWE – o czym niżej). Po pomyślnej rejestracji (czy to email, czy Google OAuth) backend tworzy rekord użytkownika (twórcy) w bazie danych i wydaje token JWT (lub ustawia cookie sesyjne HttpOnly) dla frontendu, aby utrzymać zalogowanie.

Automatyczne zakładanie portfela dla Twórcy: Każdy nowo zarejestrowany twórca otrzymuje automatycznie przydzielony portfel Circle typu Developer-Controlled Wallet (DCW) – jest to kluczowy element onboardingu. W momencie potwierdzenia rejestracji backend wywołuje API Circle (przez SDK) w celu utworzenia nowego portfela Smart Contract Account (SCA) dla tego twórcy. Identyfikatory portfela (oraz odpowiadające adresy blockchain) są zapisane w bazie TipJar. Dzięki temu tuż po rejestracji twórca ma już gotową infrastrukturę do otrzymywania napiwków – nie musi sam zakładać portfela kryptowalutowego ani konfigurować żadnych kluczy. Jest to duże usprawnienie i obniżenie bariery wejścia. Z perspektywy Circle zwiększa to liczbę aktywnych portfeli DCW i stanowi demonstrację łatwości integracji ich SDK.

Logowanie i konta Fanów: Fani – czyli osoby wspierające twórców napiwkami – na start nie muszą zakładać konta, mogą działać jako goście (co jest istotne dla niskiego progu użycia). Jednak platforma przewiduje także opcję rejestracji fana, aby umożliwić im dodatkowe funkcje (np. własny portfel do przechowywania środków, historię transakcji itp.). W Kamieniu Milowym 2 planujemy wprowadzić konta dla fanów, w tym logowanie poprzez e-mail/hasło, Google, a także Twitch (popularny wśród streamerów) oraz Sign-In with Ethereum (SIWE) dla użytkowników Web3. Dodanie logowania Twitch i SIWE wymaga rozszerzenia modułu Auth o kolejne strategie (passport-twitch, obsługa message podpisywanego portfelem dla SIWE). Po rejestracji, fan również może (opcjonalnie) otrzymać własny portfel Circle DCW zarządzany przez TipJar – analogicznie jak twórca. Dzięki temu zarejestrowani fani zyskują możliwość łatwiejszego tipowania (wewnętrznym portfelem) i przechowywania środków w aplikacji.

Profile Twórców: Każdy twórca po rejestracji tworzy swój profil publiczny, który służy jako strona docelowa dla fanów chcących wysłać napiwek. Profil zawiera nazwę twórcy, avatar, opis oraz przede wszystkim link i kod QR kierujące do możliwości dokonania wpłaty. Profile twórców są publicznie dostępne (w stylu „tipjar.com/@nick”) i generowane z wykorzystaniem SSR/SSG dla lepszego SEO. Użytkownik (fan) odwiedzając profil twórcy widzi przycisk “Wesprzyj” i prosty widget płatności, gdzie może zadeklarować kwotę napiwku w USDC i wybrać metodę płatności. Profil twórcy stanowi więc centralny element łączący fana z twórcą w naszym systemie.

Proces Wysyłania Napiwku (Tipowanie)

Proces wysyłania napiwku został zaprojektowany tak, aby był maksymalnie prosty nawet dla osoby nieznającej się na kryptowalutach. Poniższy diagram ilustruje koncepcyjnie przepływ wartości: Fan -> TipJar+ -> Twórca:

Schemat działania TipJar+: Fan przekazuje napiwek (fiat lub krypto) za pośrednictwem platformy TipJar+, a Twórca otrzymuje środki w postaci stablecoina USDC na swoim portfelu.

1. Tipowanie jako Gość (bez konta): W pierwszej fazie (MVP) zakładamy, że wielu fanów może nie chcieć zakładać konta tylko po to, by wysłać napiwek. Dlatego umożliwiamy płatność jako gość, w dwóch wariantach:

Płatność kartą płatniczą (fiat): Fan wprowadza dane karty i kwotę napiwku (np. 5 USD). Po stronie backendu wykorzystujemy Circle Payments API w trybie on-ramp – środki fiat zostają automatycznie przeliczone na USDC i zasilają portfel DCW twórcy. Dla fana jest to transparentne: wpisuje kwotę w USD, obciążamy jego kartę, a twórca otrzymuje równowartość w USDC. Dzięki temu nawet osoby nieposiadające żadnej kryptowaluty mogą łatwo wesprzeć twórcę.

Płatność kryptowalutą (Web3): Zaawansowani fani mogą wybrać opcję zapłaty kryptowalutowej. W najprostszym ujęciu otrzymują oni adres portfela USDC twórcy (lub skanują kod QR z tym adresem) i mogą wysłać USDC ze swojego portfela (np. MetaMask) bezpośrednio na ten adres. Na etapie MVP oznacza to normalną transakcję on-chain – fan musi pokryć opłatę gazu i posiadać np. MATIC/ETH w swoim portfelu (co dla mniej doświadczonych może być barierą, ale ta opcja jest głównie dla obeznanych). TipJar+ rejestruje taką wpłatę poprzez webhooki lub sprawdzając stan portfela twórcy. Uwaga: docelowo (KM3) planujemy usprawnić tę ścieżkę poprzez mechanizm Circle Paymaster, tak aby fan płacący z portfela web3 nie musiał posiadać natywnego tokena na gas – opłata zostanie pobrana w USDC (o czym dalej).

Niezależnie od metody, gdy napiwek dotrze do portfela twórcy (czy to wewnętrznym transferem, czy on-chain), backend:

odnotowuje transakcję w swojej bazie (model Transaction w Prisma, z informacją od kogo, dla kogo, kwotą, datą, metodą itp.),

wysyła odpowiedź do frontendu potwierdzającą powodzenie transakcji, by UI mogło np. wyświetlić komunikat “Dziękujemy za napiwek!”,

(jeśli zaimplementowane) generuje w czasie rzeczywistym powiadomienie dla twórcy, że otrzymał napiwek.

2. Tipowanie z konta TipJar (fan zarejestrowany): W kolejnych etapach rozwoju (KM2) wprowadzamy możliwość założenia konta fana i tym samym posiadania przez fana wewnętrznego portfela TipJar (Circle DCW). Zarejestrowany fan może zasilić swój portfel TipJar+ (np. depozytem USDC przez kartę lub przelewem krypto) i następnie wysłać napiwki z tego wewnętrznego salda. Taki proces jest jeszcze wygodniejszy i szybszy: fan np. jednym kliknięciem potwierdza wysłanie 5 USDC ze swojego salda na portfel wybranego twórcy. Transfer odbywa się całkowicie w ramach Circle (DCW → DCW), co oznacza brak

opłat i natychmiastowe zasilenie portfela twórcy. W aplikacji dodajemy dedykowaną stronę portfela fana z podglądem salda, historią transakcji oraz opcją doładowania środków (via karta). Fan z portfelem TipJar nie musi już każdorazowo podawać danych płatności – ma środki w aplikacji, które może rozdysponować między różnych twórców. To zwiększa jego zaangażowanie i wygodę (a dla nas oznacza więcej aktywnych portfeli i transakcji w ekosystemie Circle).

3. Tipowanie zewnętrznym portfelem bez opłat gas (gasless): Najbardziej zaawansowany scenariusz, planowany w ramach KM3, to pełne wyeliminowanie problemu opłat sieciowych dla fanów korzystających z własnych portfeli kryptowalutowych. Osiągniemy to poprzez integrację Circle Paymaster. W praktyce działa to tak:

Fan wybiera w aplikacji opcję “Zapłać krypto (bez opłat gas)”. W tle TipJar+ rejestruje, że transakcja zostanie obsłużona przez Paymaster.

Fan łączy swój portfel (np. MetaMask) i inicjuje transakcję przesłania USDC na adres portfela twórcy w sieci Polygon. Jednak dzięki mechanizmowi Account Abstraction portfele SCA twórców mogą akceptować meta-transakcje, gdzie opłata za gas jest pokrywana przez stronę trzecią.

Circle Paymaster (usługa v0.8) pozwala sfinansować gas tą trzecią stroną, a konkretnie umożliwia fanowi zapłacić za gas w USDC zamiast natywną kryptowalutą. W praktyce platforma TipJar zleca Paymasterowi opłacenie gasu, pobierając równowartość opłaty z portfela USDC fana.

Dla fana oznacza to, że nie potrzebuje w ogóle posiadać MATIC/ETH – wystarczy, że ma USDC na napiwek plus minimalnie dodatkowe USDC na opłatę (choć możliwe, że TipJar w modelu biznesowym zdecyduje się nawet pokrywać tę opłatę promocyjnie).

Transakcja zostaje nadana on-chain, Paymaster pokrywa gas, twórca otrzymuje USDC, a fan widzi np. komunikat “Opłata sieciowa: 0.02 USDC”.

W ten sposób obniżamy jedną z największych barier użyteczności DeFi dla zwykłych użytkowników. Gas Station i Paymaster we współpracy umożliwiają, że ani twórca, ani fan nie muszą martwić się o posiadanie natywnych tokenów na opłaty sieciowe – TipJar+ i Circle biorą to na siebie w zamian za drobną opłatę w USDC. W efekcie wysłanie napiwku on-chain staje się równie proste co przelew w aplikacji bankowej.

Po stronie implementacyjnej, dodamy odpowiednie opcje w interfejsie (wybór metody płatności: karta, portfel TipJar, portfel zewnętrzny gasless), z jasną komunikacją dla użytkownika na czym polega opcja gasless. Backend będzie musiał obsłużyć zgłoszenie transakcji przez Paymaster (prawdopodobnie integracja z API lub kontraktem Circle Paymaster) oraz rozliczenie tych opłat (np. zmniejszenie salda fana o koszt gas w USDC).

Wyплаты Środków przez Twórców

TipJar+ nie zatrzymuje środków twórców – każdy twórca może w dowolnym momencie wypłacić zgromadzone napiwki ze swojego portfela TipJar (Circle DCW) na dowolny zewnętrzny portfel kryptowalutowy (np. swój własny MetaMask lub konto na giełdzie) lub potencjalnie konto bankowe (jeśli wdrożymy wypłaty fiat). Proces wypłaty jest kluczowy dla domknięcia cyklu wartości i zostanie zaimplementowany w ramach Kamienia Milowego 4.

Interfejs wypłaty (frontend): W panelu twórcy pojawi się sekcja “Wypłata środków”, gdzie twórca może wpisać kwotę USDC do wypłaty oraz adres swojego portfela zewnętrznego (format adresu będzie walidowany). Jeśli przewidujemy obsługę wielu sieci, twórca może też wybrać docelową sieć (np. Polygon, Ethereum – choć na start skupimy się na Polygon). Po zleceniu wypłaty, twórca zobaczy nowy wpis w historii wypłat ze statusem “w toku” wraz z ewentualnym identyfikatorem transakcji. Gdy transakcja się potwierdzi, status zmieni się na “zakończono” (lub “nieudane” w przypadku błędu).

Logika wypłaty (backend): Po otrzymaniu żądania wypłaty, backend (NestJS) obsługuje je następująco:

Weryfikuje uprawnienia – czy token JWT należy do danego twórcy i czy twórca ma wystarczające saldo USDC.

Jeśli ok, generuje zlecenie wypłaty poprzez SDK Circle: wywołuje Circle Transfers API lub Payouts API, zlecając transfer USDC z portfela DCW twórcy na wskazany adres zewnętrzny. Pod spodem Circle może zrealizować to jako transakcję on-chain na blockchainie (np. wysłanie USDC z naszego master konta Circle na adres docelowy).

Dzięki temu, że portfele DCW twórców są typu SCA, opłata gas za tę transakcję może zostać pokryta przez Circle Gas Station – twórca dostanie pełną kwotę, nie martwiąc się o koszty transakcyjne. TipJar może zdecydować, czy koszt gas pokrywa platforma (jako bonus dla twórców), czy potrąca odpowiednią równowartość USDC ze środków (np. wspomniane 3,5% opłaty wypłaty może zawierać koszt gas).

Backend zapisuje w bazie żądanie wypłaty i jego status (np. PENDING). Następnie poprzez webhook od Circle lub okresowe sprawdzanie statusu transakcji dowiaduje się o finalizacji. Gdy Circle potwierdzi wykonanie transferu, backend aktualizuje status wypłaty na COMPLETED i ewentualnie zmniejsza saldo twórcy (jeśli do tej pory trzymaliśmy środki wewnątrz).

W przypadku błędu (np. błędny adres, brak środków, błąd sieci) – backend oznacza wypłatę jako FAILED i przekazuje twórcy komunikat o niepowodzeniu.

Dla twórcy proces ten jest prosty: klika wypłacić, chwilę czeka i środki pojawiają się na jego koncie poza platformą. Funkcjonalność ta czyni TipJar+ kompletnym rozwiązaniem – twórcy mogą nie tylko zbierać fundusze, ale też je dowolnie dysponować poza platformą.

Powiadomienia i Panel Twórcy

Aby zwiększyć interaktywność platformy, planujemy wprowadzić powiadomienia w czasie rzeczywistym oraz rozbudowany Creator Dashboard. Gdy fan wyśle napiwek, twórca powinien natychmiast otrzymać informację (np. powiadomienie push w przeglądarce lub alert w panelu). Wdrożymy mechanizm nasłuchiwanie nowych transakcji (np. poprzez WebSocket z backendu lub integrację z usługą typu Pusher) i wysyłania powiadomień push. Powstanie także Live Feed dla streamerów – lista najnowszych napiwków pojawiająca się na bieżąco, co twórca może np. udostępnić na swoim streamie (funkcja analogiczna do “recent donations” na Twitchu). Zaimplementowanie powiadomień znacząco zwiększy zaangażowanie – zarówno twórcy, jak i inni fani od razu widzą aktywność, co może zachęcać do dalszego wspierania.

Panel Twórcy (Creator Dashboard): Będzie sukcesywnie rozbudowywany. W wersji podstawowej (KM2) zawiera tylko podstawowe statystyki (łączna kwota USDC otrzymana, liczba transakcji) oraz listę otrzymanych napiwków. W kolejnych wersjach dodamy bardziej zaawansowane wykresy i statystyki (np. zarobki w czasie, najlepsi fani, źródła ruchu), możliwość eksportu danych, ustalanie celów finansowych (np. pasek postępu “zbieram 100 USDC na nowy mikrofon”), itp. Rozbudujemy też opcje personalizacji profilu twórcy (zdjęcia, opisy, social media) oraz być może wprowadzimy mechanizmy interakcji z fanami (ankiety, podziękowania). Te usprawnienia zwiększą wartość platformy dla twórców, dostarczając im narzędzi do analizy i zarządzania swoim „skarbcem” napiwków.

Ulepszenia dla fanów: Poza samym procesem płatności, planujemy również funkcje społecznościowe dla fanów – np. odkrywanie twórców (strona eksploracji z możliwością filtrowania po kategoriach, wyszukiwarką streamerów, itp.), obserwowanie twórców (fan może “followować” twórcę w ramach platformy, aby otrzymywać powiadomienia o jego aktywności czy nowych celach). To wszystko ma na celu zwiększenie zaangażowania fanów i zachęcenie ich do regularnych powrotów na platformę.

Plan Realizacji – Etapy i Kamienie Milowe

Realizację projektu podzielono na serię Kamieni Milowych (KM), z których każdy dostarcza konkretny zestaw funkcjonalności. Poniżej przedstawiamy kolejne etapy rozwoju TipJar+ – od przygotowania MVP po uruchomienie produkcyjne – wraz z przypisanymi zadaniami do wykonania. Lista ta pełni rolę szczegółowej checklisty dla zespołu, umożliwiając śledzenie postępów i odhaczanie kolejnych zadań.

Kamień Milowy 1: Uruchomienie Rdzenia MVP – Onboarding Twórców i Podstawowe Napiwki

Cel: Dostarczenie minimalnej, ale w pełni funkcjonalnej wersji platformy (MVP), umożliwiającej twórcom rejestrację, automatyczne utworzenie portfela Circle oraz odbieranie pierwszych napiwków od fanów-gości. Ten etap weryfikuje podstawową architekturę i integrację z Circle.

Backend (NestJS) – zadania główne:

Konfiguracja środowiska i projektu: Przygotować środowisko deweloperskie (np. WSL na Windows), repozytorium Git, struktury katalogów. Zainicjalizować aplikację backend

(NestJS) oraz frontend (Next.js), skonfigurować bazę danych PostgreSQL (np. kontener Docker) i ORM Prisma.

Moduł uwierzytelniania (Auth): Zaimplementować rejestrację i logowanie twórców przez Email/Hasło oraz przez Google OAuth2 (Passport.js). Zaimplementować wydawanie tokenów JWT i mechanizm odświeżania (refresh token) dla utrzymania sesji użytkownika.

Moduł użytkowników i profili: Stworzyć modele danych (Prisma) dla użytkownika (User), profilu twórcy (Profile), powiązań OAuth/Social (SocialConnection), portfela Circle (CircleWallet) itp.. Zaimplementować podstawowe operacje CRUD dla profilu twórcy (w tym ustawianie nazwy, avatara, opisu).

Integracja z Circle – portfele: Włączyć moduł integracji z Circle Programmable Wallets. Przy rejestracji twórcy: automatycznie utworzyć portfel Circle DCW (SCA) i zapisać jego circleWalletId oraz mainWalletAddress w bazie. Wykorzystać oficjalny SDK Circle do komunikacji z API. Upewnić się, że sekret API Circle jest bezpiecznie przechowywany (np. w .env lub AWS Secrets Manager).

Endpointy transakcji (tip): Przygotować podstawowe endpointy do obsługi otrzymywania napiwków – np. /tips do rejestrowania nowej transakcji (wywoływany po webhooku lub przez front w zależności od metody płatności). Na razie minimalna logika: zapis do bazy, ewentualne podstawowe walidacje (czy kwota > 0, itp.).

Frontend (Next.js) – zadania główne:

Strona główna i rejestracja twórcy: Zaprojektować prosty landing page prezentujący platformę (pitch dla twórców, zalety, CTA do rejestracji). Stworzyć formularz rejestracji/logowania dla twórców (email+hasło oraz przycisk “Zaloguj przez Google”). Walidować podstawowe błędy (np. zajęty email).

Profil twórcy (publiczny): Zaimplementować stronę profilową twórcy z unikalnym URL (np. /[username]). Powinna wyświetlać nazwę twórcy, avatar, opis oraz formularz do wysłania napiwku. Formularz dla fana-gościa: pole kwoty, wybór metody płatności (przycisk “Kartą” i ewentualnie “Krypto”). Wygenerować link i kod QR do tej strony, aby twórca mógł go udostępnić.

Obsługa płatności kartą (front): Dla płatności kartą zintegrować np. Stripe Elements lub prosty formularz danych karty, który przekaże dane do backendu (bezpiecznie, zgodnie ze standardem PCI). Ewentualnie skorzystać z gotowego widgetu Circle Payments (jeśli dostępny). Po sukcesie płatności wyświetlać potwierdzenie.

Podstawowa obsługa stanu aplikacji: Po zalogowaniu twórcy, front powinien przechowywać token JWT i dodawać go do kolejnych requestów (np. w headerze Authorization). Zaimplementować prosty kontekst użytkownika (czy jest zalogowany, dane profilu itp.).

Płatności i integracje (gość):

Obsługa płatności kartą (backend): Wykorzystać Circle Payments API (konto partnerskie) do utworzenia checkoutu lub transakcji płatniczej na żadaną kwotę. Po potwierdzeniu przez procesor płatności (webhook/odpowiedź), dodać środki w USDC na portfel twórcy (wywołując wewnętrzny transfer w Circle z konta naszego projektu na portfel twórcy).

Obsługa płatności krypto (backend): Wygenerować endpoint lub mechanizm pobierania adresu portfela twórcy. Gdy fan zażąda adresu do wpłaty, zwracamy mainWalletAddress twórcy (to jest adres kontraktu SCA). W dokumentacji dla użytkowników jasno komunikujemy, na której sieci i jaki token jest wspierany (Polygon, USDC). Opcjonalnie, zaimplementować prosty monitor nowych transakcji na tych adresach (np. cyklicznie odpytywać Circle API o historię lub nasłuchiwać webhooków) – aby automatycznie wykryć otrzymane on-chain wpłaty i zarejestrować je jako tip.

Powiadomienia email: (Opcjonalnie) wysyłać twórcy powiadomienie e-mail, że otrzymał napiwek od użytkownika X w kwocie Y (można to zrealizować na bazie usługi typu SendGrid).

> Checkpoint KM1: Po ukończeniu tego etapu powinniśmy mieć działające podstawy platformy: twórca może się zarejestrować i otrzymać swój portfel; fan może wesprzeć twórcę poprzez stronę profilu, płacąc kartą lub krypto; twórca zobaczy środki na swoim portfelu i w bazie pojawi się zapis transakcji. MVP w tym zakresie pozwoli przetestować kluczową ścieżkę: fan → USDC → twórca, co jest istotnym dowodem dla interesariuszy (np. Circle) potwierdzającym, że integracja działa.

Kamień Milowy 2: Wzbogacenie Doświadczenia Użytkownika (Fan) i Rozszerzenie Integracji Circle

Cel: Rozbudowa platformy o funkcje dedykowane zarejestrowanym fanom oraz poszerzenie metod logowania. Wprowadzone zostaną konta i portfele dla fanów, nowe opcje uwierzytelniania (Twitch, SIWE) oraz podstawowy panel twórcy do podglądu statystyk. Rozszerzymy także wykorzystanie usług Circle (depozyty kartą na portfel fana, transfery między portfelami fan<>twórcą).

Backend – nowe zadania:

Nowe metody logowania: Dodać obsługę logowania przez Twitch OAuth2 (Passport strategy, klucze API Twitch) oraz przez Ethereum (SIWE). W przypadku SIWE: endpoint /auth/siwe/nonce (generuje nonce i przechowuje tymczasowo), endpoint /auth/siwe/verify (przyjmuje podpisany komunikat od użytkownika, weryfikuje go i loguje/zakłada konto fana). Wymaga integracji z biblioteką ethers.js lub viem po stronie frontendu do podpisywania wiadomości.

Konta fanów i portfele: Rozszerzyć model użytkownika o typ/rolę (fan vs twórca). Zaimplementować rejestrację fanów (analogicznie do twórców, ale prostszy profil).

Opcjonalnie: przy rejestracji fana tworzyć dla niego portfel Circle DCW – tę decyzję pozostawiamy jako konfigurowalną (może nie każdy fan będzie chciał mieć portfel od razu). Jeśli portfel fana jest tworzony, zapisywać jego ID w bazie i traktować go podobnie jak portfel twórcy.

API portfela fana: Stworzyć endpointy pozwalające frontowi pobrać saldo USDC fana (z Circle API) oraz historię transakcji fana (jego doładowań i wysłanych tipów). Dodać endpoint do inicjowania depozytu na portfel fana kartą – analogicznie do napiwku kartą, lecz tu odbiorcą jest portfel fana. Wykorzystać Circle Payments API do zasilenia portfela DCW fana.

API Dashboardu Twórcy: Dodać endpointy zwracające zestawienia statystyk (np. sumaryczna kwota napiwków, liczba fanów wspierających) oraz pełną historię otrzymanych transakcji do wyświetlenia w dashboardzie. Te dane można wyliczać na bieżąco z bazy (np. sumy, agregaty) lub trzymać pre-liczone w osobnej kolekcji.

Frontend – nowe zadania:

Strona portfela fana: Zaimplementować nową sekcję dla zalogowanego fana: “Mój portfel”. Powinna pokazywać saldo USDC, adres portfela depozytowego (jeśli fan ma DCW) oraz listę transakcji (otrzymane środki – np. własne depozyty – i wysłane napiwki). Kluczowe komponenty: WalletOverview (saldo, przycisk “Doładuj”), TransactionList (historia z datami, kwotami, typem), DepositModal (formularz doładowania kartą).

Doładowanie portfela fana: Na stronie portfela dodać przycisk “Doładuj środki” – po kliknięciu otwiera modal z integracją płatności kartą (podobnie jak w KM1, tylko że środki idą do portfela fana). Po sukcesie odświeżyć saldo i historię.

Nowe loginy UI: Dodać przyciski/logotypy “Zaloguj przez Twitch” oraz “SIWE” w formularzu logowania. Dla SIWE – po kliknięciu wywołać `ethereum.request({method:'eth_requestAccounts'})`, podpisać komunikat (biblioteką `ethers/viem`) zawierający nonce od backendu, przesłać podpis do weryfikacji.

Creator Dashboard (wersja 1): Rozbudować panel twórcy o podstronę “Dashboard” wyświetlającą podstawowe statystyki: sumę otrzymanych USDC, liczbę napiwków, oraz listę transakcji (napiwków) z podstawowymi szczegółami. To wymaga pobrania danych z nowych endpointów backendu. UI może pokazywać np. tabelkę transakcji i kilka wyróżników na górze (sumy).

Płatności (fan z kontem):

Transfery wewnętrzne: Umożliwić, by zarejestrowany fan mógł wysłać napiwek bezpośrednio ze swojego portfela TipJar. To oznacza wykonanie transferu DCW → DCW poprzez API Circle: z portfela fana na portfel twórcy. Należy zaimplementować w backendzie operację: sprawdź saldo fana -> wywołaj transfer w Circle (wewnętrzny, natychmiastowy) -> zapisz transakcję. Frontend: w widżecie płatności, jeśli fan jest zalogowany i ma środki, pokazać opcję “Zapłać z mojego portfela TipJar”.

Aktualizacja historii: Po udanej transakcji między portfelami (fana i twórcy) zaktualizować historię fana (dodanie wpisu “wysłano X USDC do Y”) oraz twórcy (otrzymano X od fana Z). To może być zrobione analogicznie do obsługi napiwku w KM1, tylko inny typ źródła.

> Checkpoint KM2: Na tym etapie platforma stanie się bardziej angażująca: fani mogą tworzyć konta, gromadzić środki i tipować wygodniej, twórcy mają podstawowy wgląd w statystyki. Rozszerzyliśmy integrację z Circle – teraz obsługujemy wiele portfeli (dla fanów), wewnętrzne transfery, oraz szersze spektrum metod logowania (co pomoże przyciągnąć np. streamerów Twitch). To dobry moment na testy użyteczności – zebrać feedback od pierwszych użytkowników odnośnie procesu rejestracji fana, używania portfela itp.

Kamień Milowy 3: Zaawansowana Elastyczność Płatności i Abstrakcja Opłat Gazowych

Cel: Wdrożenie mechanizmów eliminujących tarcia związane z opłatami transakcyjnymi na blockchain. Obejmuje to integrację Circle Paymaster dla fanów płacących z własnych portfeli (EOA), pełne wykorzystanie Circle Gas Station dla wszystkich transakcji z portfeli SCA oraz dalsze usprawnienia płatności. Na tym etapie TipJar+ stanie się praktycznie “gasless” dla użytkowników, co będzie unikalnym atutem platformy.

Zadania techniczne:

Integracja Circle Paymaster: Skonfigurować usługę Paymaster (v0.8) w naszym środowisku. Uzyskać potrzebne dane/klucze od Circle. W backendzie dodać logikę przyjmującą żądanie płatności z portfela zewnętrznego: generować tzw. UserOp (Ethereum Transaction Payload w standardzie ERC-4337) i wysłać ją do endpointu Paymastera, bądź korzystać z SDK jeśli Circle takie dostarcza. W skrócie – implementacja, która pozwoli fanom EOA płacić za gaz USDC. Należy zadbać o zabezpieczenia (np. limit kwoty, aby ktoś nie nadużył mechanizmu) oraz obsługiwać ewentualne błędy (np. gdy fan ma za mało USDC na pokrycie gas).

Integracja Gas Station (pełna): Upewnić się, że wszystkie transakcje on-chain inicjowane z naszych portfeli SCA (np. wypłaty twórców, transfery DCW → EOA) są automatycznie sponsorowane gazem. W praktyce wymaga to włączenia w ustawieniach portfeli Circle trybu Gas Station. Prawdopodobnie Circle już to zapewnia out-of-the-box dla SCA, ale trzeba przetestować. Dodatkowo, monitorować koszty – wprowadzić mechanizmy kontrolne, by rejestrować ile USDC zużywamy na pokrycie gazu (przydatne dla modelu biznesowego).

UI/UX dla gasless: W interfejsie dodać wyraźne oznaczenia “Brak opłat za gas” przy odpowiednich metodach płatności. Zapewnić, że użytkownik rozumie, co się dzieje (np. tooltip “Przy tej metodzie opłata sieci zostanie automatycznie pokryta przez platformę, możesz zapłacić gaz w USDC”). To ważne z perspektywy edukacyjnej i marketingowej.

Testy end-to-end tych mechanizmów: Ze względu na złożoność Paymastera i Gas Station, konieczne będzie gruntowne przetestowanie scenariuszy: fan z MetaMask wysyła tip bez ETH – czy na pewno dochodzi? czy gaz pobrało z USDC? co jeśli gas wyszedł droższy niż

miał USDC? Ltd. Ten kamień będzie wymagał bliskiej współpracy z Circle (to nowe technologie, być może jeszcze w wersji beta).

Inne funkcje KM3:

Ulepszenia płatności tradycyjnych: Rozważyć dodanie obsługi Apple Pay/Google Pay poprzez Circle (ułatwienie dla mobilnych użytkowników).

Waluty fiat: Ewentualnie dodać więcej walut w płatności kartą (jeśli Circle pozwala, np. płatność w EUR przeliczana na USDC).

Refactoring transakcji: Ponieważ mamy już kilka rodzajów tipów (karta -> DCW, DCW -> DCW, EOA -> DCW z gasless), warto ujednolicić i refaktoryzować kod obsługi transakcji, by był łatwo rozbudowywalny. Być może wprowadzimy pole "method" w modelu Transaction (np. CARD, INTERNAL, WEB3_GASLESS, WEB3_DIRECT).

Monitoring i alerty: Włączyć dodatkowe logowanie/monitoring dla procesów płatności (np. jakie transakcje gasless się dzieją, czy nie ma błędów Paymastera). Możliwe, że integracja z Sentry posłuży do wychwycenia ewentualnych problemów.

> Checkpoint KM3: Po zakończeniu tego etapu TipJar+ będzie oferować najnowocześniejsze rozwiązania Web3, które wyróżnią platformę: pełna abstrakcja opłat transakcyjnych. Użytkownicy nie muszą rozumieć pojęcia "gas" – dla nich działa to po prostu jak zwykła płatność. To ogromna przewaga konkurencyjna, możliwa dzięki zdecydowaniu się na Circle i Account Abstraction. Z biznesowego punktu widzenia, TipJar+ stanie się świetnym case study dla Circle Paymaster i Gas Station, co wzmocni relację z Circle (pokażemy, że realnie wykorzystujemy i testujemy ich najnowsze produkty). Po KM3 mamy w pełni dopracowany mechanizm płatności – od tego momentu skupimy się na dokończeniu brakujących funkcjonalności "dookoła" oraz dopracowaniu jakości.

Kamień Milowy 4: Pełna Funkcjonalność Wypłat oraz Dopracowanie Platformy

Cel: Umożliwienie twórcom wypłacania środków z platformy oraz dodanie brakujących funkcji ułatwiających korzystanie z TipJar+ na co dzień. Ten etap skupia się na wypłatach twórców, systemie powiadomień w czasie rzeczywistym oraz przygotowaniu platformy na większy ruch (optymalizacje, skalowanie).

Wypłaty USDC (Twórcy):

Frontend – formularz wypłaty: W panelu twórcy dodać sekcję "Wypłać środki". Pola: kwota do wypłaty, adres portfela (z automatycznym formatowaniem i walidacją), wybór sieci (o ile dotyczy). Przycisk "Zleć wypłatę". Po wysłaniu – dodanie wpisu w "Historii wypłat" z datą i statusem (np. ikona ładowania pólki w toku).

Backend – endpoint wypłaty: Zaimplementować endpoint POST /withdrawals wymagający autoryzacji twórcy. Obsługa jak opisana w sekcji wypłat: sprawdzenie salda, wywołanie Circle Payouts/Transfers API z poleceniem wysłania USDC na adres docelowy, utworzenie rekordu wypłaty w bazie (status PENDING).

Obsługa webhooku/statusu: Skonfigurować webhook od Circle informujący o statusie payoutu, lub alternatywnie w tle (cron) sprawdzać co pewien czas status nierozliczonych wypłat. Po potwierdzeniu – ustawić status na COMPLETED, odjąć środki z salda twórcy itp. Jeśli failed – oznaczyć jako nieudane i ewentualnie przywrócić środki (jeśli zdążyliśmy zablokować).

Gas Station for payouts: Upewnić się, że Gas Station pokrywa opłatę – co powinno działać, bo portfel twórcy to SCA. Test: wypłata np. 50 USDC, twórca powinien dostać ~50 USDC on-chain, a opłata w MATIC/ETH powinna zejść z naszego konta Gas Station.

Powiadomienia e-mail: (Opcjonalnie) Wysłać twórcy maila potwierdzającego wypłatę (lub alert jeśli nieudana).

Powiadomienia w czasie rzeczywistym:

Backend – mechanizm push: Wybrać technologię do powiadomień push (np. Firebase Cloud Messaging dla powiadomień mobilnych/web push, albo użyć biblioteki jak socket.io dla powiadomień w aplikacji). Na potrzeby web można użyć Web Push API (wymaga serwerowych kluczy VAPID) – może to jednak zbyt duży zakres, więc alternatywnie: Live update w aplikacji przez WebSocket.

Implementacja Live Feed: Stworzyć na backendzie serwis, który po odebraniu nowego napiwku emituje zdarzenie do nasłuchujących klientów (np. kanał “tips” z identyfikatorem twórcy). Twórca zalogowany w swoim panelu ma otwarte połączenie WebSocket i odbiera te eventy, co pozwala np. zaktualizować listę transakcji na żywo i wyświetlić powiadomienie w UI.

Powiadomienia push (opcjonalnie): Jeśli czas pozwoli, zintegrować powiadomienia push webowe: po zalogowaniu zapytać twórcę o zgodę na notyfikacje. Przy nowym tipie, używając Notification API, wyświetlić natychmiast info (“Nowy napiwek od Fan123: 5 USDC!”). To wymaga serwera push (można skorzystać z gotowych rozwiązań, np. OneSignal, albo budować samemu).

Testy UX: Sprawdzić, czy powiadomienia nie są natrętne, dodać opcję wyciszenia itp.

Optymalizacje i skalowanie:

Ładowanie danych: Zaimplementować paginację tam, gdzie brak (np. historia transakcji twórcy – zamiast zwracać całość, stronicować wyniki lub lazy-load). Poprawi to wydajność przy rosnącej liczbie rekordów.

Optymalizacje zapytań: Przeanalizować zapytania do bazy (Prisma) i do API Circle – wprowadzić cache tam gdzie sens (np. cache salda co 1 min zamiast za każdym odświeżeniem).

Przygotowanie na ruch: Skonfigurować środowisko staging do testów wydajnościowych. Przeprowadzić testy obciążeniowe kluczowych endpointów (tipowanie, wypłata) np. używając JMeter/K6, aby znaleźć ewentualne wąskie gardła. Zmierzymy, czy obecna infrastruktura wytrzyma np. 100 równoczesnych transakcji na minutę i gdzie są limity (API Circle ma rate limit – uwzględnić to).

Poprawki UX/UI: Dopracować interfejs użytkownika zgodnie z feedbackiem – np. komunikaty błędów (czytelne informacje, gdy coś pójdzie nie tak: “Spróbuj ponownie za chwilę”, “Sprawdź poprawność adresu” itd.), wskazówki dla nowych użytkowników (tooltips, onboarding guide).

Kompletność funkcji: Upewnić się, że wszystkie kluczowe funkcjonalności z dokumentacji UI/UX zostały zaadresowane – np. czy jest możliwość edycji profilu twórcy, zmiany hasła, resetu hasła przez email (to też ważne – system resetowania hasła może wymagać integracji np. z Amazon SES lub innej usługi email), itp.

> Checkpoint KM4: Po tym etapie TipJar+ jest funkcjonalnie kompletny: twórcy mogą zarabiać i wypłacać pieniądze, platforma informuje na bieżąco o aktywności, a całość działa płynnie. Mamy za sobą implementację wszystkich rdzeniowych elementów. Teraz nastąpi czas na intensywne testy, utwardzanie bezpieczeństwa i dopracowanie szczegółów przed publicznym launch.

Kamień Milowy 5: Gotowość Przedpremierowa – Testy i Bezpieczeństwo

Cel: Zapewnienie najwyższej jakości, bezpieczeństwa i stabilności platformy przed upublicznieniem. Obejmuje to kompleksowe testy (automatyczne i manualne), audyty bezpieczeństwa oraz finalizację aspektów prawnych/regulacyjnych. Po tym etapie powinniśmy być pewni, że TipJar+ jest gotowy na start.

Testowanie kompleksowe:

Testy jednostkowe i integracyjne: Pokryć testami automatycznymi krytyczne moduły backendu (np. logika tworzenia portfela, mechanizmy płatności – można mockować API Circle) oraz kluczowe funkcje frontendu (np. komponent wysyłania napiwku). Zweryfikować poprawność działania wszystkich ścieżek biznesowych.

Testy end-to-end (E2E): Przygotować scenariusze E2E symulujące rzeczywiste korzystanie: rejestracja twórcy, zalogowanie, udostępnienie profilu; fan wchodzi na profil, wysyła napiwek jako gość kartą; inny fan rejestruje się, doładowuje konto, wysyła tip; twórca sprawdza dashboard i zleca wypłatę itd.. Te testy można częściowo automatyzować (np.

Cypress/Selenium dla frontendu, skrócone symulacje dla backendu), a częściowo wykonać ręcznie z udziałem ekipy testującej.

Testy wydajnościowe: Uruchomić testy obciążeniowe – np. 1000 równoczesnych tipów w ciągu 5 minut – obserwować zużycie CPU/RAM backendu, czas odpowiedzi, ewentualne błędy. Sprawdzić zachowanie bazy danych pod obciążeniem (czy nie dochodzi do deadlocków, czy indeksy działają wydajnie). Na podstawie wyników dostosować parametry (skalowanie instancji, zwiększenie pooli, dodanie indeksów).

Testy UX (użyteczności): Zaprosić kilkunastu beta-testerów (np. zaprzyjaźnionych twórców i fanów) do wypróbowania platformy. Zebrać ich opinie na temat łatwości użycia, niezrozumiałych elementów, błędów interfejsu. Na tej podstawie wprowadzić ostatnie poprawki interfejsu (teksty, kolory, układ).

Bezpieczeństwo:

Przegląd wewnętrzny kodu: Przeprowadzić internal code review wszystkich newralgicznych fragmentów – autoryzacja (czy na pewno tylko właściciel profilu może wypłacać swoje środki, itp.), walidacje danych wejściowych, komunikacja z Circle (czy poprawnie traktujemy webhooks, nie przyjmujemy niepodpisanych requestów).

Audyty zewnętrzne: Zlecić profesjonalny audyt bezpieczeństwa firmie zewnętrznej. Powinni przeprowadzić testy penetracyjne całej aplikacji (frontend, backend, API) oraz sprawdzić konfigurację infrastruktury (np. czy bazy nie da się z zewnątrz połączyć, czy klucze API są dobrze chronione). Szczególny nacisk na integracje z Circle – czy klucze API Circle są bezpieczne, czy nie da się podmienić żądania transakcji itp. Audyt obejmie też standardowe ataki webowe (OWASP Top 10) – SQL Injection, XSS, CSRF, etc. Wyniki audytu posłużą do załatania wszelkich wykrytych luk.

Ostateczne poprawki i test regresji: Po zaadresowaniu uwag z testów i audytu, przetestować raz jeszcze krytyczne ścieżki (regresja), aby upewnić się, że poprawki nie wprowadziły nowych problemów.

Zgodność prawna i regulacyjna:

Sprawdzić, czy wszystkie polityki prywatności, regulaminy są przygotowane. TipJar+ operuje finansami – upewnić się, czy w jurysdykcji działania nie są wymagane np. licencje pieniądza elektronicznego, lub czy partnerstwo z Circle to pokrywa (prawdopodobnie Circle jako licencjonowany podmiot obsługuje custody USDC, więc TipJar może nie potrzebować własnych licencji, ale to do potwierdzenia).

Dostosować platformę do RODO jeśli planujemy użytkowników z UE (miejsce hostingu danych – już jest w EU, mechanizm usunięcia konta na żądanie, polityka cookies itp.).

Upewnić się, że marka, logo, nazwa są zarejestrowane lub przynajmniej nie kolidują z innymi.

> Checkpoint KM5: Po tym etapie powinniśmy mieć wysokie zaufanie do stabilności i bezpieczeństwa TipJar+. Wszystkie funkcje są przetestowane w boju, zaudytowane i dopracowane. Platforma jest gotowa na publiczny debiut. To także czas na działania typu bug bounty (ewentualnie ogłosić mały program nagród za znalezienie błędu, aby społeczność przetestowała) oraz finalne szlify przed wypuszczeniem w świat.

Kamień Milowy 6: Uruchomienie MVP i Monitorowanie Wzrostu

Cel: Publiczny launch TipJar+ oraz ustanowienie mechanizmów monitoringu i dalszego rozwoju po starcie. Chociaż formalnie uruchomienie to nie koniec prac, traktujemy je jako kamień milowy oznaczający przejście z fazy budowy do fazy operacyjnej.

Launch produktu:

Deployment produkcyjny: Uruchomić ostatnią wersję frontendu (deploy na Vercel z domeną produkcyjną) oraz backendu (na AWS, w trybie wysokiej dostępności). Upewnić się, że zmienne środowiskowe są ustawione na produkcyjne klucze (np. API keys od Circle w trybie live, nie sandbox). Wykonać seed podstawowych danych jeśli potrzeba (np. lista obsługiwanych sieci, konfiguracje).

Monitoring i alerty: Zaimplementować monitoring produkcyjny – zbieranie logów (np. poprzez CloudWatch, Sentry), metryk (zużycie CPU, memory), śledzenie błędów. Skonfigurować alarmy (np. email/Slack) na krytyczne zdarzenia: niedostępność usługi, błędy 500, wyjątki w wypłatach itp.

Analytics użytkownika: Włączyć narzędzia analityczne by śledzić zachowanie użytkowników po starcie (oczywiście zgodnie z polityką prywatności). Może to być Google Analytics dla ruchu web, ale też własne analizy: np. ile tipów dziennie, ARPU (średnia kwota napiwku na użytkownika), konwersja odwiedzających profil na wysyłających napiwek. Te dane pomogą w podejmowaniu decyzji produktowych.

Wsparcie i moderacja: Przygotować podstawowe narzędzia administracyjne (nawet tymczasowo prosty panel admina lub bezpośredni dostęp do bazy) na okoliczność wsparcia użytkowników – np. reset hasła, blokada konta w razie nadużyć, edycja błędnych danych profilu itp.

Działania po starcie:

Zbiór feedbacku: Aktywnie monitorować opinie pierwszych użytkowników (Twitter, Discord, e-mail kontaktowy). Reagować szybko na zgłoszenia błędów czy propozycje usprawnień.

Iteracyjne ulepszenia: Ustalić listę potencjalnych szybkich poprawek po starcie – np. jeśli zauważymy, że jakaś funkcja jest nieintuicyjna, od razu planować wydanie patcha.

Skalowanie infrastruktury: W miarę napływu użytkowników, być gotowym do zwiększenia zasobów: np. uruchomienie dodatkowych instancji backendu, zwiększenie mocy bazy danych, włączenie dodatkowych regionów CDN dla frontu itp.

Marketing i wzrost: Choć to poza zakresem czysto technicznym, launch wiąże się z kampanią marketingową (co zresztą zaplanowano osobno w harmonogramie 50-dniowym). Techniczny zespół powinien być gotów na ewentualny nagły duży ruch (np. po akcji na Product Hunt czy wzmiance od influencerów).

> Checkpoint KM6: Platforma oficjalnie działa publicznie. Od tego momentu wchodzimy w tryb operacyjny – monitorujemy, utrzymujemy i skalujemy. Należy uważnie obserwować metryki wzrostu: liczba nowych twórców tygodniowo, liczba transakcji, wolumen USDC przepływający przez system. Te dane pokażą, czy osiągamy zakładane cele i gdzie ewentualnie skupić dalsze prace.

Dalsza Wizja Rozwoju (Beyond MVP)

Stworzenie i uruchomienie TipJar+ to dopiero początek. Zgodnie z długoterminową wizją, projekt ma potencjał ewolucji w pełnoprawną platformę SocialFi do roku 2034, łączącą świat finansów kryptowalutowych z ekonomią twórców. Oto niektóre kierunki rozwoju, które rozważamy w dalszej perspektywie:

Tokenizacja i ekonomia społecznościowa: Wprowadzenie własnego tokena platformy lub tokenów personalnych twórców. Twórcy mogliby emitować Creator Tokens, które fani kupowaliby i otrzymywali w zamian dostęp do unikalnych korzyści (głos w sprawie treści, udział w dochodach, prywatne kanały komunikacji itp.). Platforma mogłaby stać się załącznikiem DAO twórców – zdecentralizowanej autonomicznej organizacji, gdzie posiadacze tokenów współdecydują o rozwoju.

Rozszerzenie modelu wsparcia: Poza napiwkami jednorazowymi, dodanie opcji cyklicznych subskrypcji on-chain (odpowiednik Patreon), crowdfundingowych celów (fani składają się na konkretny cel finansowy twórcy), czy nawet escrow smart contractów gdzie twórca oferuje coś w zamian po osiągnięciu celu.

Integracje z platformami streamingowymi: Bezpośrednie połączenie TipJar+ z YouTube/Twitch – np. pojawiające się powiadomienia o tipach na streamie, boty informujące na czacie. Być może partnerstwo z tymi platformami jeśli zauważą wartość dodaną.

Wejście na inne rynki i waluty: Obsługa innych stablecoinów (EURC, lokalne stablecoiny) dla rynków poza USA. Integracja z sieciami L2, które zyskają na popularności. Być może własna instancja w sieci Lightning Network (Bitcoin) – choć to poza Circle, ale w horyzoncie dekady możliwe, że rozszerzymy się poza ekosystem USDC.

Mobilna aplikacja: Stworzenie natywnej aplikacji mobilnej dla twórców i fanów, aby ułatwić korzystanie (choć dzięki responsywności Next.js webapp już teraz powinna działać na telefonach).

Skalowanie do masowej liczby użytkowników: Plan zakłada pozyskanie nawet do 100 milionów użytkowników w 12 miesięcy – to ambitne, ale pokazuje skalę aspiracji. Technicznie oznacza to ciągłą optymalizację i korzystanie z rozwiązań chmurowych na dużą skalę (auto-scaling, microservices, być może przejście na model serverless funkcji w newralgicznych miejscach).

Monetyzacja i model biznesowy: Doprecyzowanie i ewentualne zmiany w modelu prowizji. Być może wprowadzenie freemium dla twórców (podstawowe funkcje za darmo, zaawansowane analizy za abonament). Możliwość white-labelingu platformy dla innych zastosowań mikrofinansowania.

Społeczność i metaverse: Wejście w obszary jak NFT (np. specjalne edycje NFT dla top donatorów), integracja z metaverse (wirtualne eventy, gdzie typy dają efekty specjalne), etc.

Oczywiście powyższe punkty wybiegają daleko w przyszłość. Na ten moment nasza koncentracja pozostaje na sukcesie MVP – zdobyciu pierwszych użytkowników, udowodnieniu że model działa i jest zapotrzebowanie. Fundament, który położyliśmy (elastyczna architektura oparta na API, blockchain i integracjach z Circle) zapewnia jednak solidną bazę pod te przyszłe innowacje.

Podsumowanie

TipJar+ jest zaawansowanym technologicznie projektem łączącym świat finansów kryptowalutowych z platformą dla twórców internetowych. W powyższym planie przedstawiono całościową architekturę systemu oraz bardzo szczegółowy harmonogram prac rozwojowych – od początkowego MVP po przygotowania do launchu. Kluczowe atuty projektu to ścisła integracja z infrastrukturą Circle (co pozwala oferować unikalne funkcje jak bezgasowe transakcje) oraz nacisk na doskonałe doświadczenie użytkownika (upraszczanie kryptowalut dla masowego odbiorcy).

Dla zespołu deweloperskiego niniejszy dokument stanowi kompletny przewodnik – z jednej strony pokazujący “big picture” całego systemu (diagramy architektury, opis przepływów), z drugiej rozbijający wdrożenie na konkretne zadania i check-listy. Każdy moduł i funkcja zostały uwzględnione, co powinno ułatwić podział prac, estymację oraz realizację krok po kroku. Przy zachowaniu dobrej organizacji i jakości wykonania, TipJar+ ma szansę nie tylko zadebiutować z sukcesem, ale i szybko się skalować – zarówno pod względem liczby użytkowników, jak i rozszerzanych funkcjonalności.

Na zakończenie: powodzenia dla całego zespołu w realizacji tej ambitnej wizji. TipJar+ może stać się przełomową platformą w dziedzinie wsparcia twórców – wszystko w Waszych rękach, by uczynić ją dopracowaną, bezpieczną i przyjazną dla użytkowników. Do dzieła!

Źródła i materiały referencyjne: Dokumentacja techniczna TipJar+ (analizy, specyfikacje i plany), materiały Circle (API docs, Programmable Wallets, Paymaster), wytyczne UI/UX, plan marketingowy 50-dniowy, oraz zebrane best practices z branży. Wszystkie te elementy zostały uwzględnione powyżej, aby zapewnić spójność produktu z założeniami i wymogami projektu. Powracajmy do tego dokumentu na bieżąco, aktualizując go w miarę postępów – będzie on żywym opisem rozwoju TipJar+. Powodzenia!